

Hybrid Semantic Search for IIT Institutional Scientific Documents Using SPECTER2 and BM25

Mirco Senes
mirco.senes@gmail.com

June 16, 2026

1 Details of the proposal

This proposal describes a creative NLP project that explores the use of semantic embedding models to improve scientific document retrieval, while also considering how they can be combined with traditional lexical methods. The project consists in the design and implementation of an experimental search system for the scientific publications of the Istituto Italiano di Tecnologia (IIT).

The system will first focus on semantic retrieval, with the goal of finding contextually relevant publications even when there is no exact keyword overlap between the query and the retrieved document. During the development and evaluation process, complementary lexical retrieval strategies will also be considered to address cases where exact terms, acronyms, or short technical expressions are important.

The project will involve tens of thousands of documents from the IIT institutional database. Each record will be associated with key metadata such as authors, publication year, and source details in order to enable well-organized search filtering.

A prototype web application will also be developed to allow users to:

- submit natural-language queries;
- interact with the search system;
- apply filters to refine results;
- review and inspect ranked results;
- provide explicit relevance feedback on retrieved documents.

The technical implementation will primarily use Python for document preprocessing and embedding generation. PostgreSQL will handle document storage, vector similarity matching and lexical retrieval logic, while the web application's frontend and backend will be built using JavaScript.

An evaluation phase will be carried out to evaluate the system's overall effectiveness and practical utility. The analysis will focus on understanding the possible impact of this advanced search solution in the research environment, using a combination of system performance tests and direct user feedback.

The proposal is expected to have a **hard** difficulty level due to the complex integration of semantic models, retrieval techniques, metadata filtering, and the development of a full-stack evaluation prototype.

Contents

1	Details of the proposal	1
2	Introduction	3
2.1	Project Context	3
2.2	Problem Definition	3
2.3	Objectives of the Project	3
3	Background and Related Work	3
3.1	Information Retrieval	3
3.2	Semantic Search	4
3.3	Scientific Language Models	5
3.4	SPECTER and SPECTER2	5
4	Semantic Retrieval Exploration	7
4.1	Dataset and Document Processing	7
4.2	Semantic Embedding Generation	7
4.3	Embedding Space Analysis	8
4.4	Semantic Retrieval Pipeline	9
5	Hybrid Retrieval Approach	9
5.1	Limitations of Pure Semantic Search	9
5.2	Lexical Retrieval with BM25	9
5.3	Hybrid Search Strategy	10
5.4	Metadata-Based Filtering	10
6	Experimental Analysis	11
6.1	Evaluation Methodology	11
6.2	Automated Tests	11
6.3	Discussion of Results	12
6.4	User Feedback	13
7	System Implementation	14
7.1	System Architecture and Tools	14
7.2	Web Application	14
8	Future Work	15
9	Conclusion	16

2 Introduction

2.1 Project Context

As the number of scientific publications increases, efficient information retrieval has become essential for research institutions.

The Istituto Italiano di Tecnologia (IIT) manages a large scientific production distributed across multiple research domains. To organize and access this information, the IIT Data Analysis Office developed **Scientilla**, an institutional platform designed to manage research and career-related data [1]. The platform stores scientific publications and related metadata, allowing users to retrieve documents through structured search functionalities and strict filters.

2.2 Problem Definition

With the development of the new version of Scientilla, the need emerged to improve the overall search experience offered by the platform. The current search system relies on exact title matches, which works fine for precise searches but totally misses the context. Although it provides efficient keyword-based exploration, it remains strongly dependent on exact lexical matches.

In scientific environments, similar concepts can often be expressed through different terminology, abbreviations, or domain-specific formulations. As a consequence, researchers end up missing out on a lot of relevant documents.

For this reason, the new version of Scientilla requires the integration of a more sophisticated mechanism able to merge the existing search functionalities with semantic search capabilities, enabling the use of natural language queries.

2.3 Objectives of the Project

The main objectives of this project are:

- **Implementation of an Advanced Search System:** Developing a semantic retrieval pipeline using both titles and abstracts, and analyzing how complementary retrieval methods can improve the quality of the returned research items.
- **Metadata Integration:** Preserving and integrating the structured filtering functionalities already available within Scientilla (e.g., authors, publication year, and sources) alongside the new search capabilities.
- **Prototype Development:** Building a functional web application to test the search paradigm, apply filters, and collect explicit relevance feedback from users.

3 Background and Related Work

3.1 Information Retrieval

Information retrieval helps to find relevant information within large collections of documents. In a typical information retrieval system, users express the information needed through a query, and the system returns a ranked list of documents according to their estimated relevance. This ranking process usually depends on how well the terms in the query match the terms contained in each document.

Traditional information retrieval techniques rely on lexical matching. Among these methods, TF-IDF and BM25 are widely used because they assign higher scores to documents that contain important query terms while reducing the influence of stop words, very frequent and less informative words.

Term Frequency-Inverse Document Frequency (TF-IDF) can be used to estimate how relevant a document d is for a query q . For each query term t , it assigns higher weights when t is frequent in d but rare in the overall document collection D :

$$\text{TFIDF}(t, d, D) = \text{TF}(t, d) \cdot \text{IDF}(t, D)$$

The relevance score of d for q can then be obtained by summing the TF-IDF weights of the query terms that appear in the document:

$$\text{score}(q, d) = \sum_{t \in q} \text{TFIDF}(t, d, D)$$

$\text{TF}(t, d)$ stands for Term Frequency and measures how often t appears in d , while $\text{IDF}(t, D)$, Inverse Document Frequency, increases the weight of terms that occur in fewer documents of D [2].

Despite its simplicity, TF-IDF has some limitations. In particular, it does not properly control the effect of repeated occurrences of the same term, and it does not explicitly account for document length. As a result, long documents or documents that repeat a query term many times may receive high scores even when they are not necessarily more relevant.

Best Matching 25 (BM25) addresses these limitations by extending the TF-IDF scoring idea. It still estimates the relevance of a document d for a query q by considering the query terms t , but it introduces a saturated term-frequency component and a normalization factor based on document length:

$$\text{score}_{\text{BM25}}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdL}}\right)}$$

Here, $f(t, d)$ is the frequency of term t in document d , $|d|$ is the document length, and avgdL is the average document length in the collection. The parameter k_1 controls how quickly repeated occurrences of a term stop increasing the score, while b controls the strength of document-length normalization. In this way, BM25 reduces the excessive influence of both repeated terms and very long documents.

BM25 is widely used in practical retrieval applications, including web search engines, enterprise search systems, e-commerce search and recommendation, and first-stage candidate retrieval before applying more computationally expensive ranking methods [3].

3.2 Semantic Search

Semantic search aims to retrieve documents by considering the meaning of a query rather than relying only on exact keyword matching. Instead of returning results only when the same words appear in both the query and the document, semantic search tries to capture the underlying concepts expressed by the user. This allows the system to retrieve relevant documents even when they use different terms, synonyms, abbreviations, or domain-specific expressions.

This kind of search relies on text embeddings. A text embedding is a numerical vector that represents the meaning of a word, sentence, query, or document in a continuous vector space. Early approaches such as Word2Vec focused on word embeddings, representing individual words as vectors learned from their surrounding context. More recent models, such as

BERT [4], extend this idea by producing contextual representations that can encode larger textual units, including sentences, queries, and documents. Texts with similar meanings are expected to have similar vectors, so semantic similarity can be estimated by comparing their positions in the vector space, for example using cosine similarity.

While traditional search mainly ranks documents according to lexical information such as keyword frequency and metadata, semantic search focuses on how well each document matches the intent and context of the query. In this way, the ranking is based not only on shared words, but also on the conceptual similarity between the query and the retrieved documents.

3.3 Scientific Language Models

Scientific language models are neural models designed to represent and process scientific text. They are particularly useful for semantic search because they can generate embeddings that capture both terminology and contextual meaning.

Although BERT is effective for many NLP tasks, it is a general-purpose model trained on broad-domain text. Scientific documents have specific characteristics, such as technical terminology, abbreviations, formal structure, and domain-dependent expressions. For this reason, general-purpose language models may not fully capture the vocabulary and structure of scientific literature, especially when applied to titles, abstracts, and research-oriented queries.

This limitation motivates the use of scientific language models, which adapt transformer-based architectures to scientific corpora. **SciBERT** is an example of this approach: it follows the BERT architecture but is trained on a **large collection of scientific papers** [5]. As a result, SciBERT can produce representations that are more suitable for scientific terminology and academic writing.

However, SciBERT is mainly designed to model textual content, while semantic search over scientific publications often requires document-level embeddings that capture relationships between entire papers. SPECTER addresses this limitation by representing scientific papers through both their textual content and citation relationships, so that papers connected in the research community can be mapped closer in the embedding space [6]. This makes SPECTER, and later SPECTER2, more appropriate for document-level retrieval and recommendation tasks.

3.4 SPECTER and SPECTER2

SPECTER (Scientific Paper Embeddings using Citation-informed Transformers) is a transformer-based model designed to generate dense vector representations of scientific publications. Its primary objective is to map research papers into a semantic embedding space where related documents are located close to one another, facilitating tasks such as semantic search, document recommendation, clustering, and scientific information retrieval.

From an architectural perspective, SPECTER is built upon the Transformer encoder architecture and uses SciBERT as its underlying language model. Given the title and abstract of a scientific publication, the model produces a single fixed-size embedding representing the entire document. Unlike traditional language models that are trained only on textual information, SPECTER incorporates **citation relationships** during training through a **triplet-loss** objective. As shown in Figure 1, each training example is composed of a query paper $\mathcal{P}^Q$, a related paper $\mathcal{P}^+$ that is citation-related to the query paper, and an unrelated paper $\mathcal{P}^-$ that is not citation-related. These three papers are encoded by the same Transformer initialized with SciBERT, and the resulting representations are optimized with the following objective:

$$\mathcal{L}(\mathcal{P}^Q, \mathcal{P}^+, \mathcal{P}^-) = \max(d(\mathcal{P}^Q, \mathcal{P}^+) - d(\mathcal{P}^Q, \mathcal{P}^-) + m, 0)$$

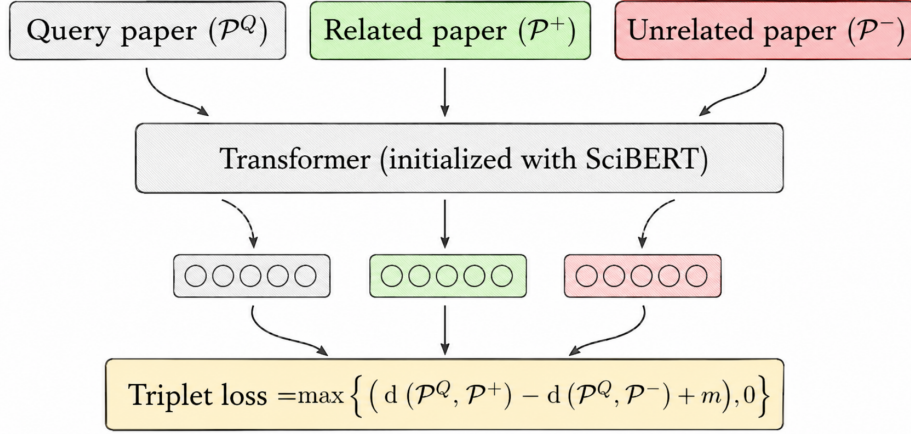


Figure 1: Overview of the triplet-loss training strategy used by SPECTER.

In this formula, $d(\mathcal{P}^Q, \mathcal{P}^+)$ is the distance between the query paper and a citation-related paper, while $d(\mathcal{P}^Q, \mathcal{P}^-)$ is the distance between the query paper and an unrelated paper. The margin m forces the positive paper to be closer to the query paper than the negative one. If this condition is already satisfied, the loss is zero; otherwise, the model is updated to reduce the distance from the positive paper and increase the distance from the negative paper. This training objective allows SPECTER to learn paper embeddings that reflect both textual similarity and citation-based scientific relatedness.

Despite its strong performance, SPECTER presents some limitations. The original model is trained using a single citation-prediction objective and produces a single type of embedding intended to serve all downstream applications. As a result, it may not generalize equally well across different scientific domains and retrieval tasks.

SPECTER2 addresses this limitation by extending SPECTER with a multi-format training strategy and **task-specific adapters**. These adapters allow the model to specialize its representations for different scientific representation tasks without fine-tuning the whole Transformer encoder [7].

The training process of SPECTER2 combines citation information with multiple supervised objectives derived from scientific literature, including citation prediction, document classification, regression, proximity-based retrieval, and ad-hoc search. The ad-hoc search format is especially important for this project because it models the case in which a short query is used to retrieve longer scientific documents. For this reason, the SPECTER2 **ad-hoc search adapter** is particularly useful for the query-encoding component of the proposed retrieval system, where users submit concise natural-language queries and expect relevant publications ranked from titles and abstracts.

Another important advantage of SPECTER2 is its broader scientific coverage. While the original SPECTER model was mainly trained and evaluated on citation information concentrated in Computer Science and biomedical literature, SPECTER2 is designed to work across a **wider set of fields of study**. This aspect is particularly relevant for the IIT publication database, because institutional scientific production can span heterogeneous domains rather than a single research area.

As a consequence, SPECTER2 produces more robust and versatile document embeddings

than the original SPECTER. Its evaluation is also supported by **SciRepEval** [7], a broader multi-format benchmark for scientific document representations that assesses different task formats and fields of study rather than relying on a narrow evaluation setting. Experimental results reported on this benchmark show improvements across several scientific representation tasks, making SPECTER2 particularly suitable for semantic search systems that must operate on heterogeneous collections of research publications. For these reasons, SPECTER2 was selected as the semantic representation model used in this project.

4 Semantic Retrieval Exploration

Before defining the final semantic retrieval pipeline, an initial exploration also considered SciBERT and the original SPECTER model. These models were useful as reference points for comparing different approaches to scientific text representation. However, for the reasons discussed in Section 3.3 and Section 3.4, the final implementation adopted SPECTER2, which is better aligned with the goal of retrieving heterogeneous IIT scientific publications from natural-language queries.

4.1 Dataset and Document Processing

The semantic retrieval pipeline started from the IIT research information database, which contains the institutional scientific outputs managed by Scientilla. The database covers heterogeneous research topics, including **Computational Sciences**, **LifeTech**, **Nanomaterials**, and **Robotics**, ranging from computational models and biomedical technologies to advanced materials and robotic platforms. For each selected record, the system retrieved the internal document identifier together with the available metadata.

A preprocessing step was then applied to obtain a cleaner textual input for the embedding model. In particular, the title and abstract were extracted from each record and normalized by removing prefixes such as *Editorial*, *Preface*, *Erratum*, and similar publication-related labels. Records without a meaningful title or abstract were discarded, while documents with a valid title but no abstract were kept, since the title could still provide useful semantic information.

The textual representation used for retrieval was built by concatenating the cleaned title and abstract when both fields were available. These choices made it possible to preserve as many valid research outputs as possible while ensuring that the semantic model received a compact and informative description of each document.

4.2 Semantic Embedding Generation

The semantic representation of each document was generated with SPECTER2. For the document embedding phase, the implementation used the `allenai/specter2.base` model version together with the SPECTER2 `proximity` adapter. In this way, the preprocessed title-and-abstract texts were encoded with an adapter specialized for proximity-based scientific document representation, rather than with the base encoder alone.

Each preprocessed document was tokenized and truncated to a maximum length of 512 tokens. The embeddings were computed in batches for efficiency, using the contextual representation of the [CLS] token from the final transformer layer as the document vector. The resulting vectors were then L2-normalized, producing fixed-size dense representations that could be compared through vector similarity.

Finally, the generated embeddings were stored in PostgreSQL using the `pgvector` extension [8]. `pgvector` is a PostgreSQL extension that adds a vector data type and vector-similarity operations, making it possible to store document embeddings in the same database as the publication metadata and retrieve nearest neighbors directly from SQL. Each vector was linked to the corresponding research item identifier. Records excluded during preprocessing were assigned a null embedding value and were not considered during semantic retrieval.

4.3 Embedding Space Analysis

To obtain a qualitative understanding of the embedding space generated by SPECTER2, an additional visual analysis was carried out using the TensorFlow Embedding Projector tool [9]. A subset of publication embeddings was exported together with the corresponding metadata and loaded into the projector, making it possible to inspect the spatial organization of the vectors through dimensionality-reduction techniques.

Figure 2 shows two **t-SNE** representations generated with the TensorFlow Projector. The left panel provides a high-level view of the embedding space and suggests that SPECTER2 tends to organize publications into semantically coherent regions, where documents with related topics appear close to one another and form distinguishable clusters. The right panel shows a more concrete example obtained with the search functionality of the projector: papers containing the word *hand* in either their title or abstract were selected and highlighted. The 50 highlighted papers tend to concentrate in a limited area of the projected space, suggesting that the model places documents related to **similar concepts close to each other**. This behavior is consistent with the expected properties of semantic embeddings and supports the idea that SPECTER2 captures meaningful relationships between scientific publications.

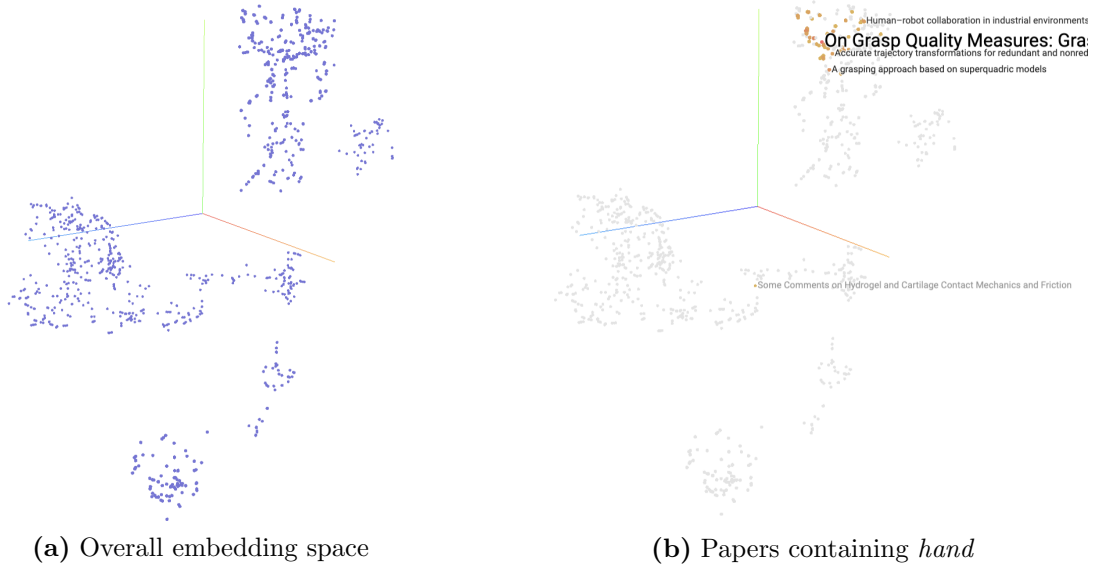


Figure 2: t-SNE visualizations of SPECTER2 publication embeddings generated with the TensorFlow Embedding Projector: (a) overall embedding space and (b) highlighted papers whose title or abstract contains the word *hand*.

Several empirical analyses were performed in this way by selecting different terms and inspecting the corresponding regions of the embedding space. These visual checks contributed to the preliminary evaluation of the model and helped confirm the advantages discussed in the background section, supporting the final choice of SPECTER2 for the semantic retrieval

component.

4.4 Semantic Retrieval Pipeline

At retrieval time, the user query is encoded separately using the SPECTER2 **ad-hoc search adapter**. This adapter is used only for query encoding, because it is designed to represent short search queries in a way that is suitable for comparison with scientific document embeddings. The normalized query vector is then compared with the document vectors stored in PostgreSQL through `pgvector`, and the closest documents are returned according to vector similarity.

5 Hybrid Retrieval Approach

5.1 Limitations of Pure Semantic Search

After the first implementation of the retrieval pipeline, the system was tested using only semantic embeddings generated with SPECTER2. This approach worked well at a conceptual level, especially when the query expressed a general research topic or described the desired document through related concepts rather than exact words. In these cases, semantic similarity made it possible to retrieve publications that were contextually close to the user’s information need, even when the query did not exactly match the title or abstract.

However, these initial tests also showed an important limitation. In several cases, users may search for highly **specific terms**, such as project names, robot names, or technical expressions that carry a very precise meaning inside the IIT research context. For example, queries containing terms such as *iCub*, the humanoid robot developed at IIT, were not always handled satisfactorily by the purely semantic approach. Even when the concept was relevant, relying only on vector similarity could fail to give enough importance to the exact occurrence of these specific terms in the indexed documents.

Another limitation was observed with very **short queries**. In some cases, different short queries repeatedly returned the same publications, often records with no abstract and only short, poorly informative titles. A plausible explanation is that both the query and these documents provided too little text to generate specific embeddings. As suggested by the embedding-space visualization in Figure 2, useful retrieval depends on placing related texts close to clear semantic regions. With very short queries, the query vector can become too generic and may not fall inside a well-defined cluster, leading the system to return results that seem reasonable in general but do not fully match the user’s intent.

These observations motivated the introduction of a hybrid retrieval strategy. The idea was to preserve the advantages of semantic search for conceptual matching, while adding a lexical component able to reward exact matches for acronyms, abbreviations, and domain-specific keywords. This was useful both for highly specific queries, where exact terms are important, and for very short queries, where the semantic representation may become too generic. For this reason, BM25 was introduced as a complementary retrieval signal and later combined with the SPECTER2-based semantic score.

5.2 Lexical Retrieval with BM25

The lexical component relies on BM25 to retrieve documents according to term matching. BM25 gives more importance to query terms that are rare in the overall collection, while reducing the impact of very common words. This makes it useful in scientific search, where a

single precise expression can strongly identify a relevant publication.

In this project, BM25 was introduced as a complementary signal to semantic retrieval. Its role is to preserve the strengths of keyword search when the exact wording of the query is important, for example with robot names, project names, experimental techniques, model names, or compact scientific expressions. In these cases, the presence of the same term in the title or abstract can be a strong indicator of relevance. BM25 is also useful for short queries, because it can still exploit exact word occurrences when the semantic embedding of the query is too generic. In the implementation, this component was handled directly inside PostgreSQL through `pg.textsearch` [10]. This extension provides BM25-based full-text search and relevance-ranked retrieval inside the database, while relying on PostgreSQL text-search configurations supporting language-dependent tokenization, stemming, and stop-word handling. The BM25 index used the extension’s default parameters, with $k_1 = 1.2$ for term-frequency saturation and $b = 0.75$ for document-length normalization.

5.3 Hybrid Search Strategy

The hybrid search strategy combines the two retrieval components instead of selecting only one of them. For each user query, the system retrieves one ranked list using SPECTER2 semantic similarity and another ranked list using BM25 lexical matching. The semantic list is useful for documents that are conceptually close to the query, while the BM25 list is useful when exact terms are important.

The two rankings are then merged using **Reciprocal Rank Fusion (RRF)**. This method does not directly compare the raw BM25 and vector similarity scores, because they are produced on different scales. Instead, it assigns a fusion score according to the position of each document in the two ranked lists. Documents that appear near the top of at least one ranking receive a higher score, and documents retrieved by both components are further promoted. In this way, the final ranking can benefit from both semantic similarity and exact lexical matching while remaining simple and robust.

5.4 Metadata-Based Filtering

Scientilla has always relied on metadata to improve the user’s search experience. In the original platform, users can refine publication search through structured information such as authors, sources, publication years, research entities, and document types. For this reason, the hybrid retrieval system was designed not to replace this behavior, but to preserve it while adding semantic and lexical ranking capabilities. The goal was to keep the search experience familiar for users, while improving the relevance of the retrieved results.

In the implemented pipeline, metadata filters are applied at database-query level. This means that the system first **restricts the set of candidate research items** according to the selected filters, and then applies the retrieval strategy only to the filtered subset. The same filtering conditions are used for both the SPECTER2 ranking and the BM25 ranking before the two lists are combined through RRF. In this way, the hybrid search respects the user’s metadata constraints while still benefiting from semantic and lexical matching.

The prototype web application includes the filters considered most useful for testing the system and closest to the original Scientilla search experience: **publication type, year range, source title, source type, authors, and keywords**. These filters allow users to narrow the search space before inspecting the final ranked results.

6 Experimental Analysis

6.1 Evaluation Methodology

The evaluation of the proposed retrieval system is based on two complementary approaches. Since the IIT publication database covers multiple specialized research domains, the first approach relies on an LLM to automatically generate queries for a controlled subset of papers. The second approach is user-centered and relies on the prototype web interface to collect explicit relevance feedback, with the idea of letting IIT researchers with domain expertise test the system and provide more meaningful conclusions. In this way, the evaluation combines reproducible system-level tests with direct evidence of practical usefulness from expert users' perspective.

6.2 Automated Tests

Since manually defining reliable queries for many heterogeneous scientific publications would have required strong domain expertise, the automated evaluation used an LLM to generate natural-language queries for a selected set of papers. The query generation step was performed using GPT-5.5. For each selected paper, the model generated two queries associated with a known target document. This made it possible to evaluate retrieval performance in a controlled way, because every query had an expected publication to retrieve.

The automated analysis started from 50 selected papers. This number was chosen as a practical compromise: it kept the LLM prompt manageable while still allowing the evaluation to cover multiple research areas. The selected papers were used to define the target documents and generate the evaluation queries, while retrieval was performed against the entire publication dataset. In this way, each query tested whether the system could retrieve the expected paper among all available IIT publications. For each query, the evaluation measured whether the expected paper appeared within the first 5, 10, and 20 retrieved results.

The selected papers cover a diverse set of scientific topics, including nanotechnology, advanced materials, robotics, biomedical sciences, and computational methods. This diversity provides a representative sample for evaluating retrieval performance across different research domains, which is important because the IIT database is not limited to a single scientific area.

The prompt used for **query generation**, together with the CSV containing the selected publications, was the following:

For each paper, generate exactly 2 search queries that a real user could write to retrieve that paper.

Input fields:

- id
- title
- abstract

Generate:

1. lexical short query with acronyms: use words that appear in the title or abstract with a very short query, preferably 2-6 tokens, using acronyms, abbreviations, or compact technical terms.
2. semantic query: use concepts, synonyms, or paraphrases, avoiding using words from the title/abstract.

Output valid JSON only.

The two generated query types were intentionally designed to stress different components of the retrieval system. For each paper, one query was lexical-like and one query was more semantic. The **lexical query** tested whether the system could retrieve the document when the

user relied on compact technical terms, acronyms, or expressions already present in the paper data. The **semantic query** instead tested whether the system could retrieve the same document when the user expressed the information need through related concepts or paraphrases, while trying not to directly reuse the words contained in the title or abstract. This distinction was important because it allowed the evaluation to compare the individual behavior of BM25 and SPECTER2, as well as the combined behavior of the hybrid retrieval strategy. Therefore, all generated queries were tested in three retrieval modes: BM25 only, SPECTER2 only, and hybrid search.

This query-generation procedure also introduces some **limitations**. The queries generated by an LLM remain an artificial approximation of real user behavior, because they were produced from the title and abstract of a known target paper and under explicit prompt constraints. Lexical queries may therefore be cleaner than real user queries, while semantic queries may be artificially difficult when the prompt asks the model to avoid title and abstract terms. Moreover, in scientific language, some technical expressions are hard to paraphrase completely, so partial lexical overlap can remain even in semantic queries. For this reason, the automated results should mainly be read as a comparative test between retrieval modes, while the feedback collected from researchers will be essential to obtain more reliable conclusions about the practical usefulness of the system.

6.3 Discussion of Results

The automated evaluation was carried out on 100 generated queries in total, corresponding to two queries for each of the 50 selected papers. All queries were searched against the full indexed publication dataset, which contains approximately 50,000 documents. The evaluation reports results for $k \in \{5, 10, 20\}$. In particular, **hit@5** measures whether the expected paper appears among the very first results, **hit@10** approximates the first page of results that a user may realistically inspect, and **hit@20** measures broader recovery over a larger result window. The analysis also includes **MRR@k**, which captures how early the first correct result appears when it is retrieved within the top k, and **Avg. rank@20**, which reports the average position of the expected paper when it is found within the top 20 results.

Mode	Cases	Hit@5	Hit@10	Hit@20	MRR@5	MRR@10	MRR@20	Avg. rank@20
Hybrid	50	0.800	0.860	0.880	0.632	0.638	0.640	2.364
SPECTER2	50	0.780	0.800	0.800	0.488	0.491	0.491	2.225
BM25	50	0.960	0.980	0.980	0.812	0.814	0.814	1.531

Table 1: Retrieval results for lexical-like queries.

Table 1 reports the results for lexical-like queries, which represent the most favorable setting for keyword-based retrieval. Since these queries include distinctive wording that often overlaps with the title or abstract of the target paper, BM25 achieves the best performance across all metrics. It reaches 0.960 hit@5, 0.980 hit@10, 0.980 hit@20, and 0.814 MRR@20, showing that exact lexical matching is highly effective when the query contains distinctive terminology. The hybrid system and SPECTER2 also obtain strong results, but they do not improve over BM25 in this setting because the lexical signal is already sufficient to identify the target documents.

Mode	Cases	Hit@5	Hit@10	Hit@20	MRR@5	MRR@10	MRR@20	Avg. rank@20
Hybrid	50	0.300	0.420	0.600	0.153	0.169	0.181	7.600
SPECTER2	50	0.300	0.420	0.540	0.148	0.163	0.171	6.741
BM25	50	0.240	0.300	0.380	0.192	0.201	0.206	5.421

Table 2: Retrieval results for semantic queries.

The results for semantic queries, shown in Table 2, are considerably lower because this setting is more difficult. These queries were designed to be broader, more conceptual, and less lexically aligned with the target papers. As a result, exact keyword overlap is weaker, and the system must rely more on semantic similarity. In this scenario, SPECTER2 retrieves more target papers than BM25 at all hit-rate cut-offs, confirming that dense representations are useful when the query and the document use different wording. The hybrid system obtains the best hit@20, reaching 0.600, because it benefits from the semantic signal while still retaining lexical evidence when available. BM25, however, keeps a higher MRR@20 among the cases it successfully retrieves. This indicates that when a semantic query still contains a few distinctive terms, BM25 may rank the correct paper relatively high, even though it recovers fewer target papers overall.

Mode	Cases	Hit@5	Hit@10	Hit@20	MRR@5	MRR@10	MRR@20	Avg. rank@20
Hybrid	100	0.550	0.640	0.740	0.393	0.404	0.410	4.486
SPECTER2	100	0.540	0.610	0.670	0.318	0.327	0.331	4.045
BM25	100	0.600	0.640	0.680	0.502	0.507	0.510	2.618

Table 3: Overall retrieval results across all generated queries.

The overall results in Table 3 combine the two query types and highlight the complementary behavior of the three retrieval modes. BM25 achieves the highest hit@5 and MRR@20, which means that it is the strongest method when the goal is to place exact matches as high as possible in the ranking. The hybrid system matches BM25 at hit@10 and achieves the best hit@20, retrieving the expected paper in 74% of the test cases. This suggests that combining BM25 and SPECTER2 improves the probability of recovering the target publication across different query styles, especially when users formulate less exact or more conceptual queries.

This interpretation also depends on the strict evaluation protocol used in the automated tests. Each query was searched against the entire institutional dataset, and only one publication was counted as correct, although other retrieved papers could still be scientifically relevant. Therefore, the automated evaluation should be interpreted as a target-document recovery test rather than a complete relevance evaluation of the ranked results. Under this setting, BM25 remains the most effective option for precise technical searches, while SPECTER2 is more useful when the query is broader and does not use the same words as the target paper. The hybrid strategy represents the most balanced solution for the proposed system, since it supports both keyword-oriented and concept-oriented search behavior within the IIT publication collection.

6.4 User Feedback

The feedback evaluation system has been implemented in the prototype web application and is currently being tested by researchers. In an earlier version, the interface allowed users to choose between BM25, SPECTER2, and hybrid retrieval. In the current version used for user testing, this choice has been removed and the system runs only in hybrid mode, making the search experience simpler and more user-friendly. Through this interface, users will be able to test the system by submitting natural-language queries, inspecting the ranked results returned by the hybrid retrieval pipeline, and providing explicit feedback on the relevance of the retrieved publications. This feedback will make it possible to assess the perceived usefulness of the deployed system and identify cases where the ranking does not satisfy users’ information needs.

For each retrieved publication, the implemented interface allows users to provide binary relevance feedback by selecting either *Relevant* or *Not Relevant*. As shown in Figure 3, these controls are integrated directly into the result inspection workflow. When users select *Not*

Relevant, the interface prompts them to specify one or more reasons for the negative judgment and optionally provide a short textual explanation. During the testing phase, users will also be able to rate the overall quality of the search session, enabling the collection of both result-level and query-level feedback. This mechanism provides a simple and intuitive way to gather expert assessments and identify situations in which the retrieved results do not adequately satisfy the user’s information need.

Figure 3: Additional feedback fields shown when users mark a retrieved publication as *Not Relevant*.

7 System Implementation

7.1 System Architecture and Tools

The system is organized with Docker as a small set of containerized services, each corresponding to one part of the hybrid search pipeline [11]. At the center of the architecture there is **PostgreSQL**, which stores the research items, their metadata and the generated document embeddings. This database layer supports both vector-based retrieval through **pgvector** and BM25 lexical retrieval through **pg_textsearch**, allowing semantic and keyword-based candidates to be retrieved close to the stored data. The same database is also used to persist user feedback in a dedicated table, storing each search session together with the original query, applied filters, returned results, global judgment, item-level relevance judgments, and timestamps.

Semantic query encoding is delegated to a dedicated **embedding service**. This service loads the SPECTER2 model and its ad-hoc search adapter, receives the user’s query from the backend, and returns the corresponding query embedding. Separating this component from the rest of the application makes the architecture clearer, because the neural model is isolated from the web API and can focus only on transforming natural-language queries into vectors suitable for semantic comparison.

The **Node.js backend** acts as the orchestration layer of the system. When a search request is received, it applies the selected metadata filters, calls the embedding service to encode the query, executes the semantic and BM25 retrieval steps against PostgreSQL, and combines the two ranked lists through the hybrid ranking strategy described in the previous sections. The **frontend**, served through a lightweight web server, provides the user-facing interface for submitting queries, applying filters, inspecting ranked publications, exploring similar items, and collecting relevance feedback.

7.2 Web Application

The prototype web application is available online at <https://scientilla-search.iit.it/>. This deployed version provides the interface through which users can interact with the hybrid

search system in a direct way. The main page allows users to write a natural-language query, apply metadata filters, inspect the ranked list of retrieved publications, and evaluate whether the returned results satisfy their information need. Each result card exposes the main bibliographic information, the abstract when available, and the retrieval scores produced by the hybrid pipeline.

A relevant interaction supported by the interface is the exploration of semantically related publications. After the ranked results are returned, the user can click on a specific paper in the list. The application then uses the selected paper as a semantic reference and displays, on the right side of the page, a list of nearby publications in the SPECTER2 embedding space. This makes it possible to continue the search beyond the initial query: instead of only inspecting the documents directly retrieved by the hybrid ranking, the user can move through the local semantic neighborhood of a relevant paper and discover other publications that are close in terms of topic, method, or research context. Figure 4 illustrates this interaction, showing the original ranked result on the left and the semantically similar items associated with the selected publication on the right.

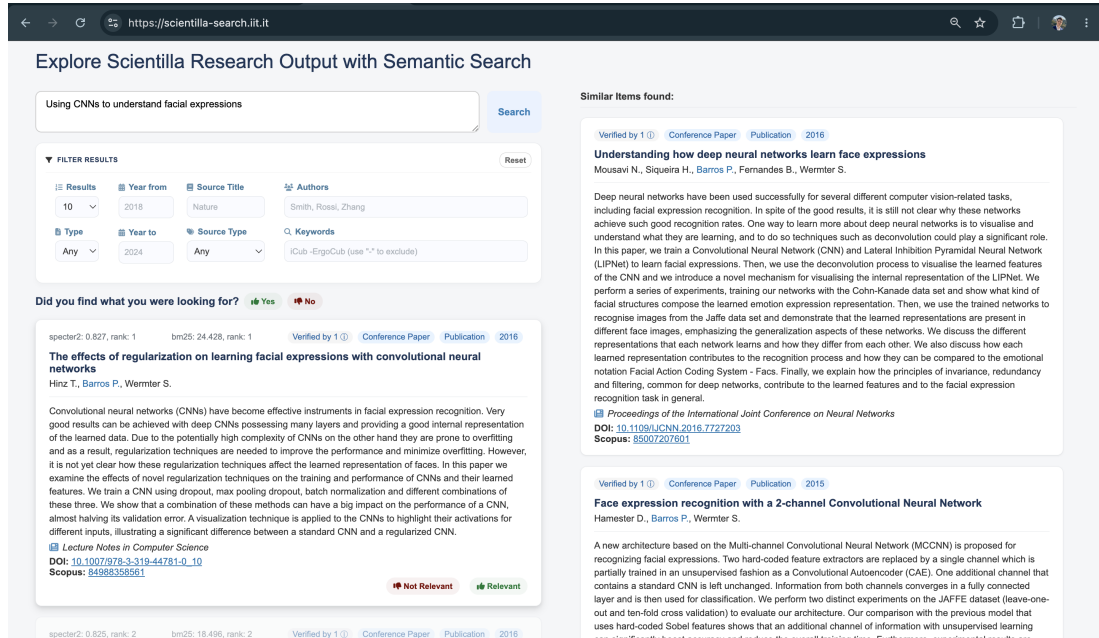


Figure 4: Web application view showing ranked search results on the left and semantically similar publications for the selected paper on the right.

8 Future Work

Future work will mainly focus on evaluating the system in real usage conditions and using the collected feedback to guide further improvements. In collaboration with the IIT Data Analysis Office, the feedback data collected through the prototype could be analyzed to identify recurring weaknesses in the retrieval process and define possible strategies for improving the ranking, the filtering behavior, and the overall search experience. This could also include testing alternative ranking or fusion strategies, in order to compare the current RRF-based hybrid ranking with different ways of combining lexical and semantic evidence.

A separate direction concerns the improvement of the automated testing procedure. Future tests could involve a larger number of documents and cover even more heterogeneous top-

ics, making the evaluation more representative of the diversity of the institutional publication database. They could also include queries containing typographical errors or misspellings, in order to assess how robust the retrieval pipeline is when users submit imperfect search inputs. These procedures could support repeated checks of the system over time, helping to verify that future changes preserve stable and reliable search behavior.

Another possible extension would be to adapt parts of the system for clustering tasks. Since the semantic representation of publications can capture relationships between research items, it could also be used to group documents by topic, research area, or methodological similarity, supporting new ways to explore the institutional publication database.

Finally, a key future step will be the integration of the prototype into the actual Scientilla platform. This will require adapting the current implementation to the production environment and to the operational requirements of the institutional system.

9 Conclusion

This project addressed the limitations of exact-match search in Scientilla by designing and implementing a hybrid retrieval pipeline for IIT institutional scientific publications. The system combines BM25 lexical ranking with SPECTER2-based semantic embeddings, so that it can handle both queries based on precise technical terms and queries expressed through broader concepts or paraphrases. At the same time, it preserves the practical requirements of the institutional setting, including metadata filtering, ranked result inspection, exploration of similar publications, and relevance feedback collection through a web prototype.

The automated evaluation over the entire indexed publication dataset, containing on the order of 50,000 documents, showed that the different retrieval approaches are useful in different situations. BM25 provided the strongest immediate quality, especially for lexical-like queries, as reflected by hit@5 and MRR@20. The hybrid strategy instead achieved the best hit@20 overall and on semantic queries. This suggests that combining lexical and semantic evidence is useful when the goal is not only to rank exact matches very high, but also to recover the target publication across different query styles.

The implemented prototype also demonstrated that the retrieval pipeline can be integrated into a complete web application, including metadata filtering, result inspection, similar-publication exploration, and relevance feedback collection. Overall, the project shows that a hybrid approach is a promising direction for improving scientific document search in Scientilla. At the same time, further testing with real users and future integration work will be important to validate the system in the institutional environment and refine it according to researchers' needs.

References

- [1] Istituto Italiano di Tecnologia. Scientilla. URL: <https://scientilla.iit.it/>.
- [2] GeeksforGeeks. Understanding tf-idf (term frequency-inverse document frequency), 2025. URL: <https://www.geeksforgeeks.org/machine-learning/understanding-tf-idf-term-frequency-inverse-document-frequency/>.
- [3] GeeksforGeeks. What is bm25 (best matching 25) algorithm, 2025. URL: <https://www.geeksforgeeks.org/nlp/what-is-bm25-best-matching-25-algorithm/>.

- [4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019.
- [5] Iz Beltagy, Kyle Lo, and Arman Cohan. Scibert: A pretrained language model for scientific text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 2019.
- [6] Arman Cohan, Sergey Feldman, Iz Beltagy, Doug Downey, and Daniel Weld. Specter: Document-level representation learning using citation-informed transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020. URL: <https://aclanthology.org/2020.acl-main.207/>.
- [7] Amanpreet Singh, Mike D’Arcy, Arman Cohan, Doug Downey, and Sergey Feldman. Scirepeval: A multi-format benchmark for scientific document representations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL: <https://aclanthology.org/2023.emnlp-main.338/>.
- [8] pgvector. pgvector: Open-source vector similarity search for postgresql. URL: <https://github.com/pgvector/pgvector>.
- [9] TensorFlow. Embedding projector: Visualization of high-dimensional data. URL: <https://projector.tensorflow.org/>.
- [10] Timescale. timescale/pg_textsearch: Postgresql extension for bm25 relevance-ranked full-text search. URL: https://github.com/timescale/pg_textsearch.
- [11] Mirco Senes. Scientilla ai: Hybrid semantic search. URL: https://github.com/mircosenes/scientilla_ai/tree/local-no-traefik.